

Computational Thinking & Coding —Week 2 Course Material

Computational Thinking & Coding ·Week 2 course material ·Seoul Theological University

One 3-hour session per week × 15 weeks ·All materials in English ·For non-major beginners Source curriculum: MIT OCW 6.100L *Introduction to CS and Programming Using Python* (Fall 2022, Ana Bell) — Lectures 3–4, Recitation 2 **Week 2 scope:** while and for loops ·looping over strings ·guess-and-check (exhaustive enumeration) **Bring to class:** the pre-class code you ran (see §3), and your prediction of its output

Learning objectives — what you can do at the end of Week 2

#	Objective
1	Say in one sentence when a while loop is the right tool and when a for loop is
2	Read a loop and predict how many times its body runs , before running it
3	Recognise an infinite loop and stop it (CTRL-C), and say what made it infinite
4	Use <code>range(start, stop, step)</code> and explain why stop is not included
5	Loop over the characters of a string and trace a counter variable pass by pass
6	Solve a problem by guess-and-check , and say when that strategy must stop

1. What we learn this week [note §...]

This week programs learn to **repeat**. Think of Netflix bingeing a show: “are there more episodes? yes → play the next one” — that is exactly a while loop, and the version that auto-plays only 4 episodes is a for loop over a fixed sequence. We will loop over numbers and strings, and use loops to solve problems by systematically **guessing and checking** every candidate answer.

2. Terms to know [note §...]

Term	One-line definition
while loop	Repeats its code block as long as a condition is True; if the condition never becomes False, it loops forever
infinite loop	A loop whose condition never becomes False (e.g. <code>while True:</code>); stop it with CTRL-C in the shell
for loop	Repeats once for each value in a sequence; the loop variable takes each value in order
<code>range(start, stop, step)</code>	Generates a sequence of ints from start up to but not including stop — a lot like string slicing
<code>break</code>	Immediately exits the innermost loop it is in, skipping the rest of the block

Term	One-line definition
guess-and-check (exhaustive enumeration)	Guess a value, check if it is correct, and keep guessing systematically until you find it or run out of values

3. Live coding — guess-and-check square root (note §...)

From Lecture 4, “Guess-and-check square root with while loop” (slide 18). Students were asked to predict the output before running it — first with 16, then with 15.

```
guess = 0
x = int(input("Enter an integer: "))
while guess**2 < x:
    guess = guess + 1
if guess**2 == x:
    print("Square root of", x, "is", guess)
else:
    print(x, "is not a perfect square")
```

Two questions to put on the board:

- How many times does the loop body run for $x = 16$?
- What happens if you enter a negative number?

Trace table to fill in during class (board work):

pass	guess before	guess**2	guess**2 < x ?	guess after
1				
2				
3				
4				
5				

4. Lab B (Recitation-style, 40 min) — ground rule (note §...)

The TA never gives away answers — the TA only asks tracing questions.

Warm-up (about 5 min)

Original Practice Problem 1: take your name as input and output the length of your name minus 5. Fill in the blanks to complete it.

```
name = ____("Enter name: ")      # (1) built-in function that reads user input
print(____(name) - ____ )        # (2) function returning string length (3) the number to subtract
```

Run it with your own name. What number is printed, and why? _____

5. Core exercises (about 30 min) — Exercise 1: remove the nth character from a string (original Problem 2) (note §...)

(a) Worked example — read this original solution, then run it.

```
test_string = "We want to remove the nth character from this string"
n = 8
new_string = test_string[:n-1] + test_string[n:]
print(test_string)
print(new_string)
```

(b) One-line change — change only `n = 8` to `n = 3`. Predict which character disappears before running: _____

(c) Fill in the blank — complete the slicing so the `n`th character is removed.

```
new_string = test_string[:____] + test_string[____:]
```

TA tracing questions

1. “With `n = 8`, what exactly is in `test_string[:7]` at this point — how many characters?”
2. “Why `n-1` on the left slice but `n` on the right? What happens if you use `n` on both?”

6. Exercise 2 — length greater than 10 or less than 5? (original Problem 3) [note §...]

(a) Worked example — read and run the original solution.

```
my_string = "This is my string"    # example string - modify to test
if len(my_string) > 10 or len(my_string) < 5:
    print(True)
else:
    print(False)
```

(b) One-line change — change `my_string` to `"Hi"`. Predict the output before running: _____

(c) Fill in the blank — complete the condition.

```
if len(my_string) ____ 10 ____ len(my_string) < 5:
    print(True)
```

TA tracing questions

1. “For `my_string = 'This is my string'`, what does `len(my_string)` evaluate to right now?”
2. “Which side of the `or` is `True` for `'Hi'`? Does Python even need to check the other side?”

7. Exercise 3 — count the letter e with a for loop (original Problem 4) [note §...]

(a) Worked example — read and run the original solution.

```
my_string = "How many times is the letter e in this string?"
number_of_es = 0                # this will be our counter
for letter in my_string:
    if letter == "e":
        number_of_es += 1
print(number_of_es)
```

(b) One-line change — change the comparison to count `"t"` instead of `"e"`. Predict the count before running: _____

(c) Fill in the blank — complete the loop.

```
number_of_es = 0
for ____ in my_string:
    if letter == ____:
        number_of_es ____ 1
```

TA tracing questions

1. “After the loop has processed the word `'times'`, what value is in `number_of_es`?”
2. “What does `letter` hold on the very first pass of the loop? And on the last?”

8. Self-check (about 5 min) [note §...]

- I can slice a string with `[ :k]` and `[k :]` and predict exactly which characters each piece holds.
- I can predict the value of an `or` condition without running the code.
- I can trace a counter variable through a `for` loop, pass by pass.

9. Questions to think about [note §...]

1. Guess-and-check “can’t test an infinite number of values — you have to stop at some point.” How do you decide when to stop searching for something in real life (a lost item, an apartment, a job)?
2. The same factorial problem can be written with a `while` loop or a `for` loop. If both work, what might make a programmer prefer one over the other?